# SimplePay app/SoftPOS – Interapp operations

2024. 09. 11.

# Table of contents

Table of contents ..... 2
1. Call the SimplePay application ..... 4
1.1. Setting Androidmanifest ..... 4
1.2. Call the deeplink ..... 5
2. Starting a payment transaction ..... 6
2.1. Required parameters to start a transaction ..... 6
2.2. Example of starting a transaction ..... 9
2.3. Returning the payment result to the merchant application ..... 10
2.4. Payment result parameters ..... 10
3. Start a refund transaction ..... 12
3.1. Required parameters to start a refund transaction ..... 12
3.2. Example of initiating a refund ..... 13
3.3. Return the refund result tot he merchant application ..... 13
3.4. Refund result parameters ..... 15
4. Start login ..... 16
4.1. Call the deeplink ..... 16
4.2. Required parameters to start interapp login ..... 16
4.3. Returning the login result tot he merchant application ..... 17
4.4. Login result parameters ..... 18
5. Start terminal registration ..... 19
5.1. Call the deeplink ..... 19
5.2. Required parameters to start terminal registration ..... 19
5.3. Returning the result of terminal registration tot he merchant application ..... 19
5.4. Terminal registration result parameters ..... 19
6. Overview of screen processes between different applications (partner application, SimplePay application and SoftPos application) in case of interapp service ..... 21
7. Transaction statuses ..... 22
8. Transaction status bank code (responseCode) ..... 26

**Document history**

| Date | Version | Changes |
|---|---|---|
| 2023. 03. 01. | 230301 | Expansion of original version to 4.2.1. version features |
| 2023. 05. 30. | 230530 | Expansion of original version to 4.2.2. version features |
| 2023. 07. 28. | 230728 | Expansion of original version to 4.2.3. version features |
| 2024. 01. 17. | 240117 | Expansion of original version to 4.2.4. version features |
| 2024. 04. 05. | 240405 | New brand name modifications |
| 2024. 09. 11. | 240911 | Enable qvik NFC |

**Changes**

2 Starting a payment transaction

New payMethod: NFC_WIRE
In the case of QR_WIRE payMethod, the displayed QR codes were generated via qvik, and can be read with any Hungarian banking application.

8 Error codes : New error codes have been added

5372 - The transfer was sent directly to your bank account, no refund is possible. Return the amount by bank transfer.
5029 - The transaction cannot be refunded yet, try again later!
7777 - NFC payment is not available
7778 - NFC is not enabled

## 1. Call the SimplePay application

SimplePay app can be called using DeepLink.
**Important: this feature is only available from version 4.2.0 of the SimplePay application!**

### 1.1. Setting Androidmanifest

In the AndroidManifest.xml file we need to set some parameters.

1. Enter the following query in the manifest so that the application can find the SimplePay application.

Info: https://developer.android.com/training/package-visibility/declaring

```
<queries>
   <package android:name="com.otpmobil.simplebusiness" />
</queries>
```

```
<!-- TODO: https://developer.android.com/training/package-visibility/declaring -->
<queries>
    <package android:name="com.otpmobil.simplebusiness" />
</queries>
```

2. Because of the backRef parameter, it is neccessary to set an intent-filter so that the result of the transaction can be sent back by the SimplePay application.
   The data values can be modified as desired.

Example:

```
<data
   android:host="transaction"
   android:scheme="myapplication"
   android:pathPrefix="/status"
/>
```

```
<!-- TODO: BACKREF -->
<activity
    android:name=".ResultActivity"
    android:exported="true">

    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data
            android:host="yourservice"
            android:scheme="yourapp"
            android:pathPrefix="/status"/>

    </intent-filter>
</activity>
```

## 1.2. Call the deeplink

We need to compile a URL to start the transaction

The URI should be:
For **Live** application: **simplebusiness://externaltransaction**
For **SANDBOX** application: **simplesbuiness-sandbox://externaltransaction**

In the URL you will need to enter two parameters, one is "**action**" and the other is "**details**".

The „action" can take two values:
- **pay**
- **refund**

And "details" is a BASE64 encoded JSON string of parameters related to he given „action".
The BASE64 encoding type should be **android.util.Base64.DEFAULT**.

Example:
**simplebusiness://externaltransaction?action=pay&details=Base64 enkódolt JSON String**

```
private val deeplinkUri: Uri.Builder
    get() = Uri.parse( uriString: "simplebusiness://externaltransaction").buildUpon()

private fun generateDeepLinkIntent(action: String, jsonString: String): Intent {
    val deeplink = deeplinkUri.apply {  this: Uri.Builder
        appendQueryParameter("action", action)
        val base64EncodedJson =
            Base64.encodeToString(jsonString.encodeToByteArray(), Base64.DEFAULT)
        appendQueryParameter("details", base64EncodedJson)
    }.build()
    return Intent(Intent.ACTION_VIEW, deeplink)
}

private fun openSimpleBusinessViaDeepLink(
    action: String,
    jsonString: String
) {
    val intent = generateDeepLinkIntent(action, jsonString)
    if (isInterappTransactionSafeToStart(intent)) {
        this@MainActivity.startActivity(intent)
    } else {
        Toast.makeText(
            context: this@MainActivity,
            text: "This feature is available from version 4.2.0 of the SimpleBusiness app.\n" +
                    "Install or update SimpleBusiness!",
            Toast.LENGTH_SHORT
        ).show()
    }
}

private fun isInterappTransactionSafeToStart(intent: Intent): Boolean {
    return packageManager.queryIntentActivities(intent, 0).size > 0
}
```

## 2. Starting a payment transaction

### 2.1. Required parameters to start a transaction

1. The total amount of the transaction
2. Order merchant ID
3. Buyer's e-mail address
4. Items
5. Language
6. Currency
7. Send receipt
8. Type of payment
9. BackRef

**1. Total amount of the transaction:**
The total amount of the payment or refund you want to initiate.

Key: "total"
Type: DOUBLE
Example: 123.00
Check: Must be a positive, floating-point number with a maximum of two decimal places. Currently, we only accept transactions based on HUF, so the amount must be rounded to a whole number.
Field type: Either total or items are mandatory. If the items array is filled, the total will not be taken into account, its value will be calculated based ont he items array.

**2. Order merchant identifier:**
The identifier of the order found in the merchant's system.

Key: „orderRef"
Type: STRING
Example: „ABC123"
Check: It can be a minimum of 1 and a maximum of 32 characters long and can only contain unaccented uppercase and lowercase letters, numbers and the following special characters: .-/()#.
Field type: REQUIRED

**3. Buyer's email address:**
The e-mail address belonging to the buyer.

Key: „customerEmail"
Type: STRING
Example: „your-customer@emailaddress.com"
Check: Must be a valid email format
Field type: OPTIONAL
Default value: Empty

**4. Items:**
The array of elements for which the transactions is to be started.

Key: „items"
Type: Array<Item>
Item:
- Product identifier – Key: „ref" - Type: STRING, Field type: OPTIONAL
- Product name – Key: „title" - Type: STRING, Field type: REQUIRED, max. 200 characters
- Product description – Key: „desc" - Type: STRING, Field type OPTIONAL

- Ordered quantity – Key: „amount" - Type: DOUBLE, Field type: REQUIRED
- Product unit price – Key: „price" - Type: DOUBLE, Field type: REQUIRED
    - the value in the field cannot be negative, zero or empty
- Product VAT - Key: „tax" - Típus: DOUBLE, Mező típusa: OPTIONAL
    - if filled in, the value in the filed must be between 0 and 100

Check: If filled, it must contain at least one element
Field type: Either total or items are mandatory. If the items array is filled, the total will not be taken into account, its value will be calculated based on the items array.
Default value: Empty

**5. Language:**
The language of the transaction

Key: „language"
Type: STRING
Example: „HU", „EN", „FR", etc.
Check: It must correspond to international country codes
Field type: OPTIONAL
Default value: „HU"

**6. Currency:**
The currency in which the transaction is initiated.

Key: „currency"
Type: STRING
Example: „HUF"
Check: Must comply with international currency abbreviations. Currently, we can only accept transactions based on HUF.
Field type: OPTIONAL
Default value: „HUF"

**7. Send receipt:**
Is it necessary to send a receipt after the transaction is completed.

Key: „needsReceipt"
Type: BOOLEAN
Example: true/false
Check: -
Field type: OPTIONAL
Default value: false

**8. Type of payment:**
The type of payment you want to initiate.
In case of LINK payMethod, the payment url is returned, it must be displayed in the partner application in some form (e.g. WebView, QR code). Available from SimplePay application version 4.2.3!
In case of QR_LINK payMethod, the link appears in the SimplePay application in the form of a QR code. Available from SimplePay application version 4.2.3!
In case of QR_CARD payMethod, the displayed QR code must be scanned with the Simple application.
In the case of QR_WIRE payMethod, the displayed QR codes were generated via qvik, and can be read with any Hungarian banking application.
In case of SOFTPOS payMethod, the touch payment screen opens through the SimplePay application.

Key: „payMethod"
Type: STRING (enum)

Possible Values: SOFTPOS, QR_CARD, QR_WIRE, QR_LINK, LINK, NFC_WIRE
Example: SOFTPOS
Check: It must match character by character, all capital letters
Field type: REQUIRED

**9. BackRef**

In the case of a DeepLink call, it is not possible to return a response, similar to the Activity Result, however, if a DeepLink is received in the request with the name "backRef", the SimplePay application can call back there and pass the result.

Key: „backRef"
Type: String
Example: yourapp://yourservice/yourParameter
Check: valid url
Field type: REQUIRED

## 2.2. Example of starting a transaction

In order to be able to send data to the SimplePay application, a JSON String must be created from the parameters specified in the merchant application, and then transferred to the SimplePay application in the manner specified in point I.

```
val payActionJson: String
    get() = """
    {
        "total": 1000.0,
        "payMethod": "SOFTPOS",
        "orderRef": "ABC123",
        "customerEmail": "your-customer@emailaddress.com",
        "items": [
            {
                "ref": "DEF123",
                "title": "Alma",
                "desc": "Pink lady",
                "amount": 3,
                "price": 225.0,
                "tax": 27.0
            },
            {
                "ref": "GHI123",
                "title": "Coca-Cola",
                "desc": "Zero",
                "amount": 6,
                "price": 398.0,
                "tax": 27.0
            },
        ],
        "language": "HU",
        "currency": "HUF",
        "needsReceipt": true,
        "backRef": "yourapp://yourservice"
    }
"""

fun startPayAction() {
    openSimpleBusinessViaDeepLink( action: "pay", payActionJson)
}
```

If we did everything correctly, we should get a link similar to this, which starts the SimplePay application with the appropriate parameters:

simplebusiness://externaltransaction?action=pay&details=IAogICAgICAgIHsKICAgIC.....

## 2.3. Returning the payment result to the merchant application

By specifying the "backRef" parameter, it is possible to send the result back to the merchant application.

The response will be contained in the "result" parameter of the specified backRef URL in BASE64 encoded form, which will be assigned by the SimplePay application.

For example:
"backRef" URL passed to merchant application:
yourapp://yourservice/yourParameter

The result returned by SimplePay:
yourapp://yourservice/yourParameter?result=IAoglCAglCAglHsKICAglC……..

When starting a transaction, the following parameters can be included in the response json:
1. „transactionId"
2. „orderRef"
3. „transactionStatus"
4. „responseCode"
5. „paymentUrl" (Available from SimplePay application version 4.2.3!)
6. „errorCode"

If the value of one of the fields is null, it will not be included in the answer.
In the case of a successful response, the fields transactionId, orderRef, transactionStatus and repsonseCode (the latter is null for transactionStatus INIT)) are included in the json.
In case of an error, only the errorCode is returned.

## 2.4. Payment result parameters

1. Transaction Simple ID
2. Merchant ID of the transaction
3. Transaction status
4. Transaction status bank code
5. Payment Url
6. Error code

**1. Transaction Simple ID:** Optional parameter, because if SimplePay checks are delayed, the process will not reach the start of the transaction.

Key: "transactionId"
Type: LONG
Example: 123456789
Field type: OPTIONAL

**2. Merchant ID of the transaction**

Key: „orderRef"
Type: STRING
Example: „ABC123"

Field type: OPTIONAL

**3. Transaction status**
In the case of LINK payMethod, it is not included in the answer!

Key: „transactionStatus"
Type: STRING
Possible Values: INIT | INPAYMENT | INFRAUD | AUTHORIZED | CANCELLED | REVERSED | TIMEOUT | FINISHED | NOTAUTHORIZED
Field type: OPTIONAL

**4.Transaction status bank code**
Available from SimplePay application version 4.2.2!
In the case of LINK payMethod, it is not included in the answer!

Key: „responseCode"
Type: STRING
Example: „2013"
Field type: OPTIONAL

**5. Payment url**
Available from SimplePay application version 4.2.3!
It is included in the answer only in the case of LINK payMethod!

Key: "paymentUrl"
Type: String
Example: „https://simplepay.hu/....."
Field type: OPTIONAL

**6. Error code**

Key: „errorCode"
Type: INTEGER
Example: 6012
Field type: OPTIONAL

## 3. Start a refund transaction

### 3.1. Required parameters to start a refund transaction

1. Original amount
2. Refund amount
3. Merchant ID of transaction to be refunded
4. Simple ID of the transaction to be refunded
5. Currency
6. BackRef

**1. Original amount:**
The amount of the original transaction on which the refund will be made.

Key: "originalTotal"
Type: DOUBLE
Example: 123.00
Check: Must match the original transaction amount
Field type: REQUIRED

**2. Refund amount:**
The amount you want to refund.

Key: „refundTotal"
Type: DOUBLE
Example: 123.00
Check: Must be greater than zero and cannot be more than the amount available for refund.
Field type: REQUIRED

**3. Merchant ID of transaction to be refunded**

Key: „orderRef"
Type: STRING
Example: „ABC123"
Check: It must match the merchant ID of the order on the original transaction.
Field type: OPTIONAL
It is not mandatory if the transactionId is specified.

**4. Simple ID of the transaction to be refunded**

Key: „transactionId"
Type: LONG
Példa: 123456789
Check: It must match the Simple ID of the order in the original transaction.
Field type: OPTIONAL
It is not mandatory if the orderRef is specified.

**5. Currency:**
The currency in which the transaction is initiated.

Key: „currency"
Type: STRING
Example: „HUF"

Check: Must comply with international currency abbreviations. Currently, we can only accept transactions based on HUF.
Field type: OPTIONAL
Alapérték: „HUF”

**6. BackRef**
In the case of a DeepLink call, it is not possible to return a response, similar to the Activity Result, however, if a DeepLink is received in the request with the name "backRef", the SimplePay application can call back there and pass the result.

Key: „backRef”
Type: String
Example: yourapp://yourservice/yourParameter
Check: valid url
Field type: REQUIRED
Alapérték: -

## 3.2. Example of initiating a refund

```
val refundActionJson: String
    get() = """
        {
            "transactionId": 123456789,
            "originalTotal": 123.00,
            "refundTotal": 123.00,
            "orderRef": "ABC123",
            "currency": "HUF",
            "backRef": "yourapp://yourservice/refund"
        }
    """

fun startRefundAction() {
    openSimpleBusinessViaDeepLink( action: "refund", refundActionJson)
}
```

In order to be able to send data to the SimplePay application, a JSON String must be created from the parameters specified in the merchant application, and then transferred to the SimplePay application in the manner specified in point I.

If we did everything correctly, we should get a link similar to this, which starts the SimplePay application with the appropriate parameters:

simplebusiness://externaltransaction?action=refund&details=IAogICAgICAgIHsKICAgIC……

## 3.3. Return the refund result tot he merchant application

By specifying the "backRef" parameter, it is possible to send the result back to the merchant application.

The answer will be contained in the "result" parameter of the given backRef URL in BASE 64 encoded form, which will be assigned by the SimplePay application.

For example:
“backRef” URL passed to merchant application:
yourapp://yourservice/yourParameter

The result returned by SimplePay:
yourapp://yourservice/yourParameter?result=IAogICAgICAgIHsKICAgIC……..

In the case of a refund, the following parameters may be included in the response json:
1. "transactionId"
2. "parentTransactionId"
3. "transactionStatus"
4. „responseCode”
5. "errorCode"

If the value of one of the fields is null, it will not be included in the answer.
In the case of a successful response, transactionId, parentTransactionId, transactionStatus and the responseCode (the latter is null for transactionStatus INIT) fields are included in the json.
In case of an error, only the errorCode is returned.

### 3.4. Refund result parameters

1. Transaction Simple ID
1. Original transaction Simple ID
3. Transaction status
4. Transaction status bank code
5. Error code

**1. Transaction Simple ID:** Optional parameter, because if SimplePay checks are delayed, the process will not reach the start of the transaction.

Key: "transactionId"
Type: LONG
Example: 123456789
Field type: OPTIONAL

**2. Original transaction Simple ID**

Key: „parentTransactionId"
Type: LONG
Example: 123456789
Field type: OPTIONAL

**3. Transaction status**

Key: „transactionStatus"
Type: STRING
Possible Values: INIT | INPAYMENT | INFRAUD | AUTHORIZED | CANCELLED | REVERSED | TIMEOUT | FINISHED | NOTAUTHORIZED
Field type: OPTIONAL

**4. Transaction status bank code**

Key: „responseCode"
Type: STRING
Example: „2013"
Field type: OPTIONAL

**5. Error code**

Key: „errorCode"
Type: INTEGER
Example: 6012
Field type: OPTIONAL

## 4. Start login

### 4.1. Call the deeplink

**important: this feature is only available from version 4.2.1 of the SimplePay application!**

We need to compile a URL to trigger the login, just like for the transaction.

The URI should be:
For **Live** application: **simplebusiness://device**
For **SANDBOX** application **simplebusiness-sandbox://device**

In the URL, you will need to enter two parameters, one is "**action**" and the other is "**details**".

The "action" can take two values:
- **set_device_name**
- **login**

**The "set_device_name" action is for registering the device in the SimplePay system, the phone receives the "deviceId" here, which performs the subsequent communication, you have nothing to do with this "deviceId" parameter.**
**The "set_device_name" also login the given user into the system, so a new login is only necessary if the user has logged out for some reason.**

**The "set_device_name" can be successfully called once with one installation, from then on it is not possible to overwrite this "deviceName" parameter, if you want to change it, you have to reinstall the application.**

**The „login" action must be used if we already have an application with a device name and want to log in.**
**After logging in, the SB app returns the result to the partner application.**

And "details" is a BASE 64 encoded JSON string consisting of parameters related to the given "action".
BASE64 encoding type should be **android.util.Base64.DEFAULT**.

### 4.2. Required parameters to start interapp login

1. Merchant email address
2. SecretKey
3. Merchant ID
4. Device name (only for SET_DEVICE_NAME)
5. BackRef

**1. Merchant email address**
The e-mail address of the given merchant.
It can be found on the "Partner data" page in the admin.

Key: „merchantEmail"
Type: STRING
Example: „your-customer@emailaddress.com"
Field type: REQUIRED

**2. SecretKey**
The secret key belonging to the given merchant.

It can be found on the "Account Manager-Technical Data" page in the SimplePay Admin system.

Key: „secretKey”
Type: String
Example: Y1o75aF2h2ut4m31P1TsTUawY8HC4U4m
Field type: REQUIRED

**3. Merchant ID**
The identifier for the given merchant.
It can be found on the "Account manager-Technical data" page in the admin.

Key: „merchantId”
Type: STRING
Example: „ABC123”
Field type: REQUIRED

**4. Device name**

Key: „deviceName”
Type: STRING
Example: Example11
Check: Minimum 4, maximum 30 characters and must not contain special characters.
Field type: OPTIONAL
It is only necessary if the SB app is called with the SET_DEVICE_NAME action.

**5. BackRef**
In the case of a DeepLink call, it is not possible to return a response, similar to the Activity Result, however, if a DeepLink is received in the request with the name "backRef", the SimplePay application can call back there and pass the result.

Key: „backRef”
Type: String
Example: yourapp://yourservice/yourParameter
Field type: REQUIRED

## 4.3. Returning the login result tot he merchant application

By specifying the “backRef” parameter, it is possible to send the result back to the merchant application.

The answer will be contained in the "result" parameter of the given backRef URL in BASE 64 encoded form, which will be assigned by the SimplePay application.

For example:
“backRef” URL passed to merchant application:
yourapp://yourservice/yourParameter
The result returned by SimplePay:
yourapp://yourservice/yourParameter?result=IAogICAgICAgIHsKICAgIC……..

In the case of a refund, the following parameters may be included in the response json:
1. "merchantId"
2. "deviceName"
3. "errorCode"

If the value of one of the fields is null, it will not be included in the answer.
In the case of a successful response, the merchantId and deviceName fields are included in the json.
In case of an error, only the errorCode.

### 4.4. Login result parameters

1. Merchant ID
2. Device name
3. Error code

**1. Merchant ID**
Key: „merchantId"
Type: STRING
Example: „ABC123"
Field type: OPTIONAL

**2. Device name**
Key: „deviceName"
Type: STRING
Example: Example11
Field type: OPTIONAL

**3. Error code**
Kulcs: „errorCode"
Típus: INTEGER
Példa: 6012
Field type: OPTIONAL

## 5. Start terminal registration

### 5.1. Call the deeplink

**important: this feature is only available from version 4.2.1 of the SimplePay application!**

We need to compile a URL to start the terminal registration, just like for the transaction.

The URI should be:
For **live** application: **simplebusiness://terminal-registration**
For **SANDBOX** application: **simplebusiness-sandbox://terminal-registration**

In the URL, it will be necessary to specify the "**details**" parameter.

An object with a "backRef" parameter is placed in "details", which must be formatted as a BASE64 encoded JSON string.
BASE64 encoding type should be **android.util.Base64.DEFAULT.**

### 5.2. Required parameters to start terminal registration

**BackRef**
In the case of a DeepLink call, it is not possible to return a response, similar to the Activity Result, however, if a DeepLink is received in the request with the name "backRef", the SimplePay application can call back there and pass the result.

Key: „backRef"
Type: String
Example: yourapp://yourservice/yourParameter
Field type: REQUIRED

### 5.3. Returning the result of terminal registration tot he merchant application

By specifying the "backRef" parameter, it is possible to send the result back to the merchant application.

The answer will be contained in the "result" parameter of the given backRef URL in BASE 64 encoded form, which will be assigned by the SimplePay application.

For example:
"backRef" URL passed to merchant application:
yourapp://yourservice/yourParameter
The result returned by SimplePay:
yourapp://yourservice/yourParameter?result=IAogICAgICAgIHsKICAgIC……..

In the case of a refund, the following parameters may be included in the response json:
1. "success"
2. "errorCode"

In the case of a successful response, only the success field is included in the json.
In case of an error, only the errorCode.

### 5.4. Terminal registration result parameters

1. Result
2. Error code

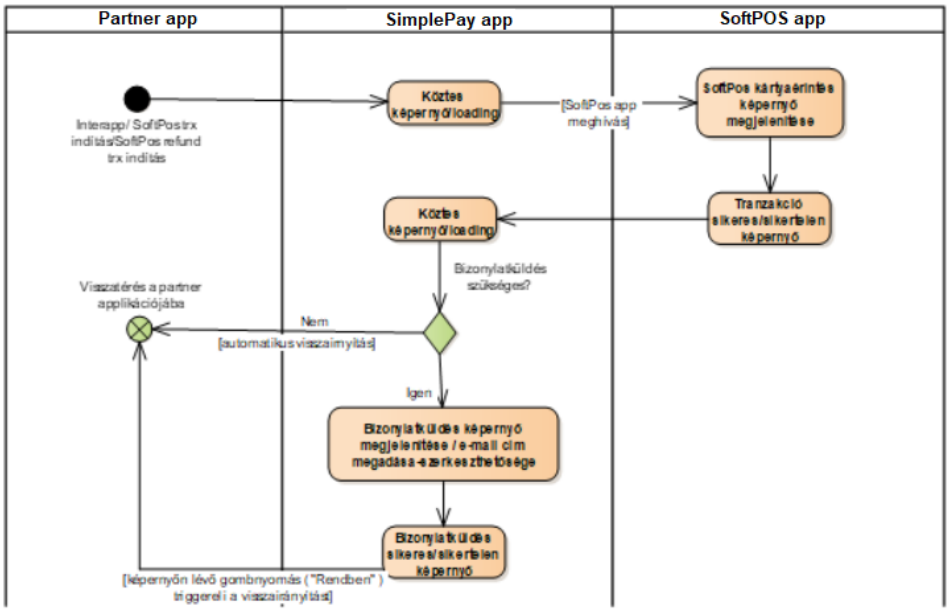
**1. Result**

Key: „success”
Type: BOOLEAN
Example: true
Field type: REQUIRED

**3. Error code**
Key: „errorCode”
Type: INTEGER
Example: 6012
Field type: OPTIONAL

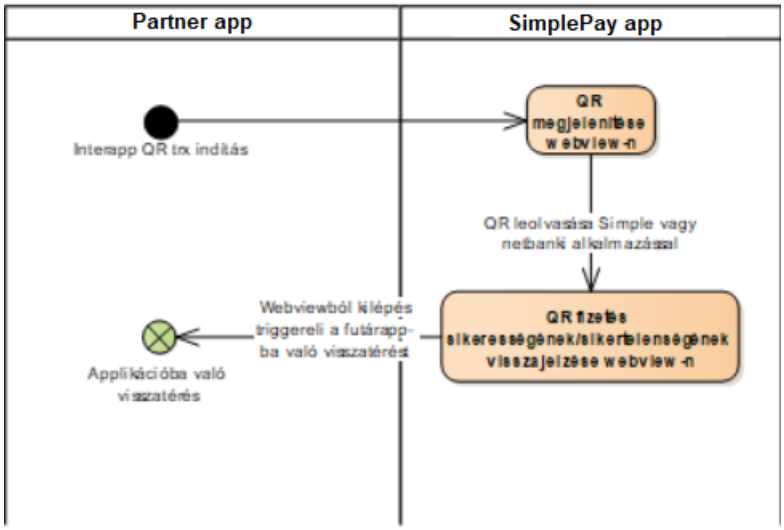
## 6. Overview of screen processes between different applications (partner application, SimplePay application and SoftPos application) in case of interapp service

SoftPos transaction initiation process (also in the case of transaction initiation and refund with a positive amount):



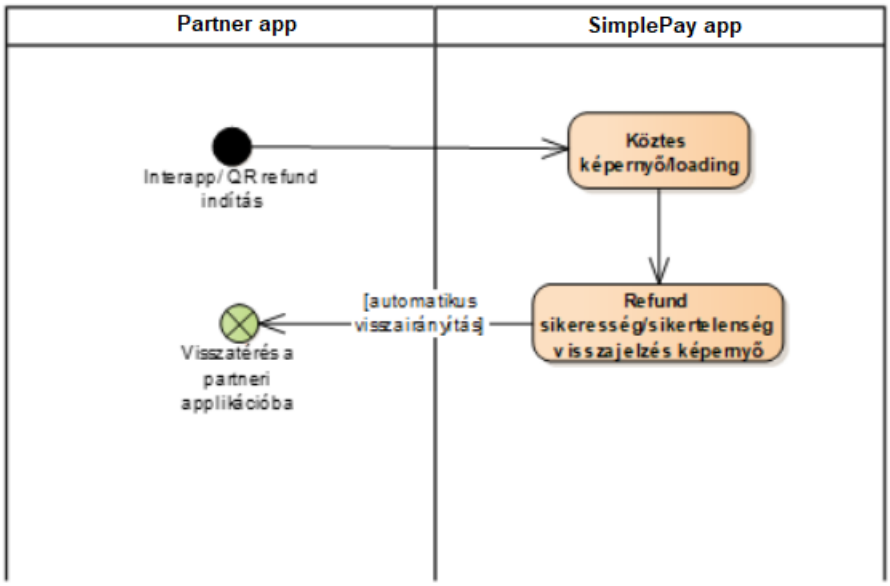

Tranzakcióról bizonylatküldés API végponton keresztül is lehetséges.

Starting a QR transaction:




Tranzakcióról e-mail értesítés automatikusan kerül kiküldésre fiók beállitástól függően.

QR transaction refund:




Tranzakcióról e-mail értesítés automatikusan kerül kiküldésre fiók beállitástól függően.

## 7. Transaction statuses

| Transaction status | Explanation | State |
|---|---|---|

| | | |
|---|---|---|
| INIT | It can only be interpreted in the case of online transactions. The transaction was created in the SimplePay system, the transaction is waiting for payment. In the case of a bank transfer, we are waiting for the transfer to arrive. | Temporary status |
| INPAYMENT | It can be interpreted in the case of online and offline transactions. During the payment of the transaction: in the online case, the "Payment" button has already been pressed on the payment interface or the QR code has been scanned and the transaction has been sent for authorization. In the offline case, we are waiting for the card to be read. | Temporary status |
| INFRAUD | In the case of online transactions, it can be interpreted as: under investigation, for the duration of fraud screening | Temporary status |
| AUTHORIZED | Can be interpreted in the case of online transactions; successful authorization after entering the card data | Failed status |
| NOTAUTHORIZED | Can be interpreted in the case of online and offline transactions; bank authorization failed | Failed status |
| CANCELLED | Can be interpreted in the case of online and offline transactions; the customer cancels the payment. | Failed status |
| REVERSED | Can be interpreted in the case of online transactions; in case of two-step transactions; reversal of blocked amount | Failed status |
| TIMEOUT | Can be interpreted in the case of online and offline transactions; transaction has expired | Failed status |
| FINISHED | Can be interpreted in the case of online and offline transactions; the transaction is successful, completed | Successful status |

Error codes

-2 – Unknown error (usually covers some kind of network error)

0 – No internet connection
401- Authentication required
403 - Forbidden
408 - Timeout

2013 - There is not enough coverage on the card
2063 - Inactive / banned card
2064, 2074 - Bad card data
2071 - Invalid card number
2072 - Missing or invalid expiration date
2073 - Invalid CVC number
2066 - Limit exceeded error
5000, 6000, 7500 - General error
5011 - The transaction cannot be found
5013 - The transaction already exists
5014 - The type of transaction submitted is not allowed on the merchant account
5015 - Refund already in progress
5016 - The transaction failed due to timeout
5017 - Transaction aborted
5018 - The transaction has already been settled
5020 - The transaction was submitted with a different amount than what was included in the original transaction
5022 - Transaction failed, refund not possible.
5023 - The currency of the refund must match the currency of the original transaction.
5024 - The email address is not correct
5026 - The transaction is prohibited
5027 - An error occurred while locking the transaction
5029 - The transaction cannot be refunded yet, try again later!
5030 - Transaction initiation is prohibited on the account.
5042 - The card is not active
5110 - The transaction cannot be refunded in the requested amount, because the pending refunds would exceed the amount of the original transaction
5201 - Invalid merchant!
5228 - The amount of refunds on a given day cannot exceed HUF 150,000
5229 - Refunds cannot be initiated for transactions older than 30 days
5305 - Failed transaction sending to the payment system
5314, 5234 - Transaction amount is not correct
5354 - The transaction was started from another device
5359 - Touch payment is temporarily unavailable
5372 - The transfer was sent directly to your bank account, no refund is possible. Return the amount by bank transfer.
5813 - Card rejected
6001 - Missing merchant data
6002 - Missing device ID
6011 - Specifying originalTotal is mandatory and must be greater than 0
6012 - Specifying refundTotal is mandatory and must be greater than 0
6024 - Deactivated device
6025 - Suspended device
6026 - Device status is incorrect
6028 - Incorrect merchant data
6029 - Unsupported version. Please update the application.
6031 - Transaction not found
6032 - The list of POS devices is empty
6035 - Missing POS device
6036 - Device status not found
6037 - Terminal status not found
6041 - The POS device is already in use

6043 - Device must be activated
6045 - The terminal must be activated
6047 - Setting terminal status failed
6048 - POS is not available
6063 - Missing bank ID
6064 - Missing IBAN number
6065 - Missing SWIFT code
6068 - Missing transaction status
6070 - Missing merchant data
6071 - Missing SoftPOS
6074 - The terminal is already registered
6075 - SoftPos is not enabled on the device
6084 - Touch payment is temporarily unavailable
6090 – Device not found
6666 - Communication error
7501 – The value of amount must be a positive number
7502 – The orderRef can be a minimum of 1 and a maximum of 32 characters long and can only contain unaccented uppercase and lowercase letters, numbers and the following special characters: .-/()#.
7503 – The value of the currency can currently only be HUF
7504 - The payMethod key can only take the following values: SOFTPOS, QR_CARD, QR_WIRE
7505 – Items must contain at least 1 element
7506 - The value of total must be greater than 0
7507 - Specifying the items or the total field is mandatory
7508 - The json passed does not conform to the expected format
7509 - The value of refundTotal must be greater than 0 and cannot be greater than the value of originalTotal
7510 - "orderRef" is invalid!
7511 - Specifying the "action" query parameter is mandatory!
7512 - Specifying the "details" query parameter is mandatory!
7513 - "transacitonId" is not correct!
7514 - Specifying the "orderRef" or "transactionId" field is mandatory!
7515 - Invalid e-mail address
7516 - Maximum allowed amount: HUF 20,000,000.
7517- Another transaction is already in progress, please try again later.
7518 - Activating touch payment may take a few minutes, please try again.
7519 - To use the touch function, please enable or download the necessary SoftPOS application from the GooglePlay store. If you have any questions, contact Customer Service.
7520 - Specifying the "backRef" parameter is mandatory!
7521 - Login is required to use this function!
7522 - The name of the product must consist of a minimum of 1 and a maximum of 200 characters!
7523 - The value of the “tax” parameter can be a minimum of 0.0 and a maximum of 100.0.
7524 - InterApp service must be enabled to use this function.
7525 - Amount above refund limit.
7526 - Invalid payment link, please try again.
7527 - SoftPos not allowed
7528 - Specifying "originalTotal" is mandatory and must be greater than 0!
7529 - Specifying "refundTotal" is mandatory and must be greater than 0!
7530 - The terminal must be activated.
7531 - Transaction amount is incorrect!
7533 - Invalid language code

7600 - Too short device name!
7601 - Device name too long!
7602 - Do not use special characters!

7603 - Another account is already assigned to the device!
7605 - Wrong backRef parameter.
7606 - User is already logged in.
7607 - A name is already set for this device.
7608 - Filling in the merchant parameter is mandatory!
7609 - Filling in the secretKey parameter is mandatory!
7610 - You already have an active terminal!
7611 - Your terminal is suspended!
7612 - Missing device ID.

7708 - Operation aborted
7709 – Transaction status update failed
7777 - NFC payment is not available
7778 - NFC is not enabled

## 8. Transaction status bank code (responseCode)

| Error code | Description | Interpreted for a softPOS transaction |
|---|---|---|
| 0 | Successful operation | x |
| 999 | Common error code. | |
| 1529 | The transaction was rejected due to invalid v/POS data | x |
| 1650 | The transaction already exists, was previously accepted (and cannot be restarted) | x |
| 2004 | Common error code | x |
| 2010 | Specified transaction type is incorrect | x |
| 2013 | Insufficient funds on the card | x |
| 2014 | Password required for payment | |
| 2016 | User canceled payment | |
| 2019 | Timeout in acquirer communication | x |
| 2020 | Technical error on the accepting bank side | x |
| 2021 | Two-factor authentication (SCA) required / PIN required | x |
| 2040 | Invalid currency | |
| 2063 | Card inactive (expired, forbidden) | x |
| 2064 | Incorrect credit card details | x |
| 2065 | Specific card involvement required / non-existent card | x |
| 2066 | Card cannot be charged / due to exceeding limit | x |
| 2068 | Invalid 3DS data / card issuer rejected 3DS authentication / general error | x |
| 2070 | Incorrect card type | x |
| 2072 | Card expiration is incorrect | |
| 2073 | The specified CVC is incorrect | |
| 2074 | Cardholder name more than 32 characters | |
| 2077 | Invalid CVC | |
| 2078 | Other error on the card issuing bank side, the card issuing bank does not provide the reason for the error | x |
| 2079 | The bank corresponding to routing is not available | x |
| 3000 | Common 3DS Error | |

| | | |
|---|---|---|
| 3001 | Invalid 3DS response message | |
| 3002 | 3DS Process Error | |